МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНО
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Омский государственный технический университет»

Кафедра «**Автоматизация и робототехника**»

Отчет по лабораторной работе №3

по дисциплине «*Программирование на языке Python»*

Проверил

доц., к.н Ситников Д.В.

_____

подпись, дата

Разработал
Студент гр. УТС-251т
Афонин С.А

_____

подпись, дата

Омск 2026

Цель работы

Изучение принципов модульной архитектуры программных систем в Python, правил импорта и создания пользовательских модулей, применения системного модуля sys и специального атрибута __name__. Освоение базовых структур данных (списков, кортежей, словарей, множеств), встроенных методов их обработки, синтаксиса срезов и применения полученных навыков при решении инженерных задач автоматизации и управления.

Краткие теоретические сведения

Модульная организация программ и системные модули

Модуль в Python представляет собой отдельный файл с расширением .py, содержащий определения функций, переменных и классов. Модульная архитектура обеспечивает повторное использование кода и декомпозицию сложных программных комплексов. Импорт модулей: Для подключения встроенных или внешних модулей используется оператор import module. Для импорта конкретных функций используется форма from module import function. Точка входа __name__: При импорте модуля весь его код исполняется один раз. Чтобы отделить код тестирования от используемого функционала, применяется точка входа с проверкой специальной переменной if __name__ == '__main__':.

Системный модуль sys: Модуль sys предоставляет доступ к параметрам среды исполнения. В частности, список sys.argv содержит аргументы командной строки, переданные скрипту при запуске.

Базовые структуры данных

Python содержит четыре основные встроенные структуры данных, предназначенные для группировки и обработки наборов элементов:

• Списки (list): Упорядоченные динамические массивы элементов произвольных типов ([]). Являются изменяемыми (mutable), что позволяет добавлять (append()), сортировать (sort()) и удалять (pop(), remove(), del) элементы. Программирование на Python: Лабораторный практикум

• Кортежи (tuple): Упорядоченные неизменяемые (immutable) последовательности элементов (()). Кортежи применяются для защиты данных от случайной модификации и возврата нескольких значений из функций.

• Словари (dict): Ассоциативные массивы, хранящие пары «ключ: значение» ({}). Ключи должны
быть уникальными и неизменяемыми объектами (строки, числа, кортежи).

• Множества (set): Неупорядоченные коллекции уникальных элементов. Множества поддерживают математические операции: объединение (|), пересечение (&), разность (-).

Индексация и операции срезов

Последовательности (списки, кортежи, строки) поддерживают обращение к элементам по индексам и получение фрагментов с помощью срезов. Синтаксис среза имеет вид [start:stop:step], где start — начальный индекс (включительно), stop — конечный индекс (не включительно), step — шаг

извлечения элементов. Отрицательные индексы: Отрицательные индексы отсчитываются от конца последовательности: - 1 соответствует последнему элементу. Разворот последовательности: Срез со значением шага -1 (например, sequence[::-1]) выполняет быстрый разворот последовательности в обратном порядке.

Системная переменная sys.argv в Python

sys.argv — это специальная переменная (список строк) из системного модуля sys в языке Python, которая содержит аргументы командной строки, переданные программе при ее запуске из консоли.

Структура и индексация списка sys.argv

• sys.argv[0] — всегда содержит имя самого запускаемого файла программного скрипта (илипуть к нему).

• sys.argv[1], sys.argv[2], ... — хранят текстовые параметры и флаги, переданные программе через пробел после её имени при консольном запуске.

Наглядный пример вызова и разбора аргументов

При выполнении в терминале следующей команды: python3 using_sys.py we are arguments. Внутри программы список sys.argv автоматически формируется из элементов.

Назначение в инженерных задачах

Использование sys.argv позволяет разрабатывать универсальные скрипты автоматизации и обработки данных, принимающие динамические параметры (например, пути к файлам замеров, пороги фильтрации, режимы работы) без необходимости редактирования исходного кода программы перед каждым запуском.

Задание. Вариант 3. Балансировка трехфазной электросети цеха.

Физический процесс / инженерное явление:

При неравномерном подключении однофазных электроприемников на предприятии возникает несимметрия токов по фазам A, B, C, приводящая к нагреву нейтрали и перегрузке трансформаторов. Необходим часовой мониторинг и выявление перекоса фаз. Задачи модуля (variant_mod.py):

1) analyze_phases(phase_loads) — принимает словарь {'A': [...], 'B': [...], 'C': [...]} и вычисляет суммарные нагрузки фаз;

2) detect_imbalance(phase_loads, max_diff=15.0) — находит максимальное расхождение средних токов и возвращает кортеж (is_imbalanced, max_diff);

3) get_last_n_samples(phase_loads, n=5) — с помощью срезов [-n:] формирует словарь с n

последними измерениями.

К программе main.py:

• Считать число n последних замеров из sys.argv[1] (дефолт 3).

• Задать почасовые токовые нагрузки, вызвать функции модуля и вывести текущее состояние сети.

Решение, 1 блок

```
def analyze_phases(phase_loads):
    """
    Вычисляет суммарные нагрузки фаз.
    """
    return {phase: sum(values) for phase, values in phase_loads.items()}


def detect_imbalance(phase_loads, max_diff=15.0):
    """
    Находит максимальное расхождение средних токов.
    Возвращает кортеж (is_imbalanced, max_diff).
    """
    averages = {
        phase: sum(values) / len(values)
        for phase, values in phase_loads.items()
    }

    difference = max(averages.values()) - min(averages.values())
    is_imbalanced = difference > max_diff

    return is_imbalanced, difference


def get_last_n_samples(phase_loads, n=5):
    """
    Формирует словарь с n последними измерениями с помощью срезов [-n:].
    """
    return {
        phase: values[-n:]
        for phase, values in phase_loads.items()
    }


if __name__ == '__main__':
    test_loads = {
        'A': [110, 115, 120, 118],
        'B': [95, 98, 92, 96],
        'C': [140, 145, 150, 148]
    }
    print(analyze_phases(test_loads))
    print(detect_imbalance(test_loads))
    print(get_last_n_samples(test_loads, n=3))
```

{'A': 463, 'B': 381, 'C': 583}
(True, 50.5)
{'A': [115, 120, 118], 'B': [98, 92, 96], 'C': [145, 150, 148]}

main

```
import sys
from variant3_mod import analyze_phases, detect_imbalance, get_last_n_samples


def main():
    n = int(sys.argv[1]) if len(sys.argv) > 1 else 3

    loads = {
        'A': [110, 115, 120, 118],
        'B': [95, 98, 92, 96],
        'C': [140, 145, 150, 148]
    }

    totals = analyze_phases(loads)
    is_imbalanced, max_diff = detect_imbalance(loads, max_diff=15.0)
    last_n = get_last_n_samples(loads, n=n)

    print("=== МОНИТОРИНГ ТРЕХФАЗНОЙ СЕТИ ===")
    print(f"Суммарные токи фаз: A={totals['A']}A, "
          f"B={totals['B']}A, C={totals['C']}A")

    status = "ДА" if is_imbalanced else "НЕТ"
    print(f"Перекос фаз: {status} "
          f"(макс. расхождение: {max_diff:.2f} A)")

    print(f"Свежие замеры (срез [-{n}:]): {last_n}")


if __name__ == '__main__':
    main()
```

=== МОНИТОРИНГ ТРЕХФАЗНОЙ СЕТИ ===
Суммарные токи фаз: A=463A, B=381A, C=583A
Перекос фаз: ДА (макс. расхождение: 50.50 A)
Свежие замеры (срез [-3:]): {'A': [115, 120, 118], 'B': [98, 92, 96], 'C': [145, 150, 148]}

Вывод

В ходе выполнения лабораторной работы №3 по дисциплине «Программирование на языке Python» (вариант 3 — «Балансировка трёхфазной электросети цеха») были в полном объёме достигнуты поставленные цели: изучены принципы модульной архитектуры программных систем, освоены правила импорта модулей, отработаны навыки работы с системным модулем sys и списком аргументов командной строки sys.argv, а также закреплены практические приёмы обработки базовых структур данных языка Python.

В процессе работы были решены следующие задачи:

1. **Освоена модульная организация программы.** Код разделён на два независимых файла: модуль variant3_mod.py, содержащий прикладные функции с обязательными docstrings, и главный исполняемый скрипт main.py, который импортирует эти функции и формирует сводный консольный отчёт.
2. **Изучена роль точки входа if __name__ == '__main__':.** Показано, как с помощью этой конструкции один и тот же файл может выступать и в роли самостоятельной программы (при прямом запуске), и в роли библиотеки функций (при импорте в другой скрипт). Реализован блок автономного тестирования модуля.
3. **Отработана работа с системным модулем sys и аргументами командной строки.**Реализовано чтение параметра n из sys.argv[1] с корректным fallback-значением по умолчанию, что позволяет запускать программу с разными настройками без редактирования исходного кода.
4. **Закреплены навыки работы с базовыми структурами данных.** В ходе решения задачи были задействованы списки (для хранения временных рядов замеров), словари (для ассоциативного хранения нагрузок по фазам), кортежи (для возврата нескольких значений из функции) и срезы последовательностей с отрицательными индексами [-n:].
5. **Проведена отладка и верификация результатов.** Программа успешно запущена как в режиме автономного тестирования модуля, так и в режиме полноценного исполнения главного скрипта. Полученные числовые результаты проверены ручным расчётом, расхождение с примером из методических указаний объяснено арифметической неточностью в самом примере.

Таким образом, в результате выполнения работы были сформированы устойчивые практические навыки построения модульных программных систем на языке Python, понимание механизмов импорта и пространств имён, а также уверенное владение базовыми структурами данных и операциями срезов. Поставленная цель достигнута в полном объёме, материал лабораторного практикума усвоен и может быть применён.